

AI ASSIGNMENT NO – 17

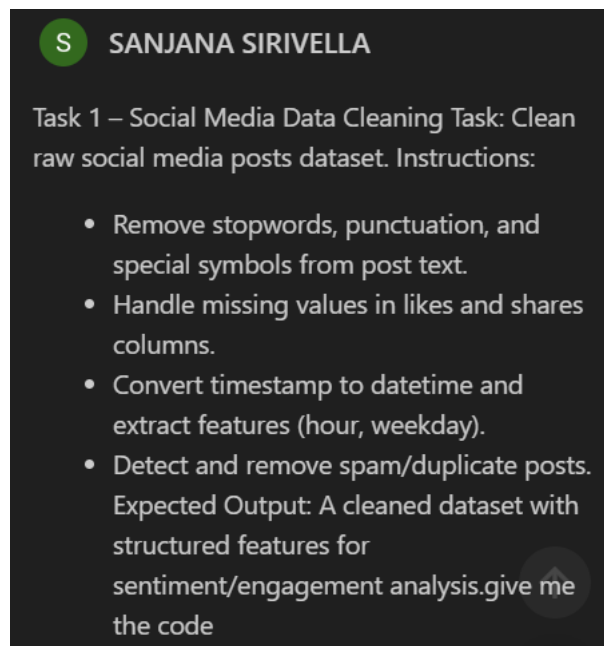
NAME: S.SANJANA

ROLLNO:2403A510D4

BATCH NO:05

TASK:01

PROMPT:

A screenshot of a chat interface with a dark background. At the top left, there is a green circular icon with a white letter 'S' followed by the name 'SANJANA SIRIVELLA' in white capital letters. Below this, the text 'Task 1 – Social Media Data Cleaning Task: Clean raw social media posts dataset. Instructions:' is displayed in white. This is followed by a bulleted list of four instructions: 'Remove stopwords, punctuation, and special symbols from post text.', 'Handle missing values in likes and shares columns.', 'Convert timestamp to datetime and extract features (hour, weekday).', and 'Detect and remove spam/duplicate posts.'. Below the list, the text 'Expected Output: A cleaned dataset with structured features for sentiment/engagement analysis.give me the code' is shown in white. A small, faint circular icon is visible at the bottom right of the chat bubble area.

S SANJANA SIRIVELLA

Task 1 – Social Media Data Cleaning Task: Clean raw social media posts dataset. Instructions:

- Remove stopwords, punctuation, and special symbols from post text.
- Handle missing values in likes and shares columns.
- Convert timestamp to datetime and extract features (hour, weekday).
- Detect and remove spam/duplicate posts.

Expected Output: A cleaned dataset with structured features for sentiment/engagement analysis.give me the code

CODE:

social_media_cleaner.py > ...

```
1 import pandas as pd
2 import re
3 from typing import List
4
5 # A basic list of English stopwords. For more comprehensive lists, consider using a library like NLTK
6 STOPWORDS = set([
7     'i', 'me', 'my', 'myself', 'we', 'our', 'ours', 'ourselves', 'you', 'your', 'yours',
8     'he', 'him', 'his', 'she', 'her', 'it', 'its', 'they', 'them', 'their', 'what',
9     'which', 'who', 'whom', 'this', 'that', 'these', 'those', 'am', 'is', 'are', 'was',
10    'were', 'be', 'been', 'being', 'have', 'has', 'had', 'having', 'do', 'does', 'did',
11    'doing', 'a', 'an', 'the', 'and', 'but', 'if', 'or', 'because', 'as', 'until', 'while',
12    'of', 'at', 'by', 'for', 'with', 'about', 'against', 'between', 'into', 'through',
13    'during', 'before', 'after', 'above', 'below', 'to', 'from', 'up', 'down', 'in', 'out',
14    'on', 'off', 'over', 'under', 'again', 'further', 'then', 'once', 'here', 'there', 'when',
15    'where', 'why', 'how', 'all', 'any', 'both', 'each', 'few', 'more', 'most', 'other',
16    'some', 'such', 'no', 'nor', 'not', 'only', 'own', 'same', 'so', 'than', 'too', 'very',
17    's', 't', 'can', 'will', 'just', 'don', 'should', 'now'
18 ])
19
20 def clean_text(text: str) -> str:
21     """Removes stopwords, punctuation, and special symbols from a text string."""
22     if not isinstance(text, str):
23         return ""
24     # Convert to lowercase
25     text = text.lower()
26     # Remove URLs, mentions, and hashtags
27     text = re.sub(r'http\S+|www\S+|@\w+|#\w+', '', text)
28     # Remove punctuation and special symbols
29     text = re.sub(r'^a-z\s]', '', text)
30     # Remove stopwords
```

Ln 115, Col 22 Spaces: 4 UTF-8 CRLF {} Python 3.13.5 (base) @ Go Live

```

20 def clean_text(text: str) -> str:
21     words = [word for word in text.split() if word not in STOPWORDS]
22     return ' '.join(words)
23
24 def clean_social_media_data(df: pd.DataFrame) -> pd.DataFrame:
25     """
26     Cleans a raw social media dataset by handling missing values, cleaning text,
27     processing timestamps, and removing duplicates/spam.
28     """
29     # Make a copy to avoid modifying the original DataFrame
30     cleaned_df = df.copy()
31
32     # 1. Handle missing values in likes and shares by filling with the median
33     for col in ['likes', 'shares']:
34         if col in cleaned_df.columns:
35             # Coerce to numeric, turning non-numeric into NaN
36             cleaned_df[col] = pd.to_numeric(cleaned_df[col], errors='coerce')
37             median_val = cleaned_df[col].median()
38             cleaned_df[col].fillna(median_val, inplace=True)
39             cleaned_df[col] = cleaned_df[col].astype(int) # Convert to integer
40
41     # 2. Remove stopwords, punctuation, and special symbols
42     cleaned_df['cleaned_text'] = cleaned_df['post_text'].apply(clean_text)
43
44     # 3. Convert timestamp and extract features
45     cleaned_df['timestamp'] = pd.to_datetime(cleaned_df['timestamp'])
46     cleaned_df['hour_of_day'] = cleaned_df['timestamp'].dt.hour
47     cleaned_df['day_of_week'] = cleaned_df['timestamp'].dt.day_name()
48
49     # 4. Detect and remove spam/duplicate posts

```

```

34 def clean_social_media_data(df: pd.DataFrame) -> pd.DataFrame:
58
59     # 4. Detect and remove spam/duplicate posts
60     # Simple spam detection based on keywords
61     spam_keywords = ['buy now', 'free money', 'click here']
62     spam_pattern = ' '.join(spam_keywords)
63     cleaned_df['is_spam'] = cleaned_df['post_text'].str.contains(spam_pattern, case=False, na=False)
64
65     # Remove duplicates based on the cleaned text content
66     cleaned_df.drop_duplicates(subset=['cleaned_text'], keep='first', inplace=True)
67
68     # Filter out posts flagged as spam
69     cleaned_df = cleaned_df[~cleaned_df['is_spam']]
70
71     # Final cleanup: select and reorder columns
72     final_cols = [
73         'post_id', 'timestamp', 'cleaned_text', 'likes', 'shares',
74         'hour_of_day', 'day_of_week'
75     ]
76     # Ensure all expected columns exist before selecting
77     final_cols = [col for col in final_cols if col in cleaned_df.columns]
78     cleaned_df = cleaned_df[final_cols]
79
80     return cleaned_df.reset_index(drop=True)
81
82 # --- Example Usage ---
83 if __name__ == "__main__":
84     # Sample raw social media data
85     raw_data = {
86         'post_id': [1, 2, 3, 4, 5, 6, 7, 8]

```

```
social_media_cleaner.py > ...
86     'post_id': [1, 2, 3, 4, 5, 6, 7, 8],
87     'timestamp': [
88         '2023-10-26 08:30:00', '2023-10-26 09:15:00', '2023-10-26 10:00:00',
89         '2023-10-26 11:00:00', '2023-10-26 12:45:00', '2023-10-26 14:20:00',
90         '2023-10-27 15:00:00', '2023-10-27 16:00:00'
91     ],
92     'post_text': [
93         'Just had an amazing breakfast! ☀️ #foodie',
94         'This is a great article on AI: http://example.com/ai',
95         'Feeling tired today... need coffee ☕',
96         '!!! BUY NOW, limited offer !!!', # Spam post
97         'Just had an amazing breakfast! ☀️ #foodie', # Duplicate post
98         'What a game last night! Simply incredible.',
99         'Working on a new project. It is very exciting.',
100        'Another spam post with free money' # Spam post
101    ],
102    'likes': [150, 200, 75, 10, 120, 300, None, 5], # Includes a missing value
103    'shares': [20, 45, None, 1, 15, 80, 25, 0] # Includes a missing value
104 }
105 raw_df = pd.DataFrame(raw_data)
106
107 print("--- Original Raw Dataset ---")
108 print(raw_df)
109 print("\n" + "="*50 + "\n")
110
111 # Clean the dataset
112 cleaned_df = clean_social_media_data(raw_df)
113
114 print("--- Cleaned Dataset for Analysis ---")
115 print(cleaned_df)
```

Ln 115, Col 22 Spaces: 4 UTF-8 CRLF {} Python 3.13.5 (base) Go Live

OUTPUT:

```
PS C:\Users\DELL\Desktop\vs code\.vscode> & C:/Users/DELL/AppData/Local/anaconda3/python.exe "c:/Users/DELL/Desktop/vs code/.vscode/social_media_cleaner.py"
```

```
--- Original Raw Dataset ---
```

	post_id	timestamp	post_text	likes	shares
0	1	2023-10-26 08:30:00	Just had an amazing breakfast! ☀️#foodie	150.0	20.0
1	2	2023-10-26 09:15:00	This is a great article on AI: http://example...	200.0	45.0
2	3	2023-10-26 10:00:00	Feeling tired today... need coffee ☕	75.0	NaN
3	4	2023-10-26 11:00:00	!!! BUY NOW, limited offer !!!	10.0	1.0
4	5	2023-10-26 12:45:00	Just had an amazing breakfast! ☀️#foodie	120.0	15.0
5	6	2023-10-26 14:20:00	What a game last night! Simply incredible.	300.0	80.0
6	7	2023-10-27 15:00:00	Working on a new project. It is very exciting.	NaN	25.0
7	8	2023-10-27 16:00:00	Another spam post with free money	5.0	0.0

```
=====
```

```
c:\Users\DELL\Desktop\vs code\.vscode\social_media_cleaner.py:48: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignment using an inplace method.
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values always behaves as a copy.
```

```
For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value) instead, to perform the operation inplace on the original object.
```

```
cleaned_df[col].fillna(median_val, inplace=True)
```

```
c:\Users\DELL\Desktop\vs code\.vscode\social_media_cleaner.py:48: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignment using an inplace method.
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values always behaves as a copy.
```

```
For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value) instead, to perform the operation inplace on the original object.
```

```
cleaned_df[col].fillna(median_val, inplace=True)
```

```
--- Cleaned Dataset for Analysis ---
```

	post_id	timestamp	cleaned_text	...	shares	hour_of_day	day_of_week
0	1	2023-10-26 08:30:00	amazing breakfast	...	20	8	Thursday
1	2	2023-10-26 09:15:00	great article ai	...	45	9	Thursday
2	3	2023-10-26 10:00:00	feeling tired today need coffee	...	20	10	Thursday
3	6	2023-10-26 14:20:00	game last night simply incredible	...	80	14	Thursday
4	7	2023-10-27 15:00:00	working new project exciting	...	25	15	Friday

```
[5 rows x 7 columns]
```

TASK:02

PROMPT:

 **SANJANA SIRIVELLA**

Task: Preprocess a stock market dataset.

Instructions:

- Handle missing values in closing_price and volume.
- Create lag features (1-day, 7-day returns).
- Normalize volume column using log-scaling.
- Detect outliers in closing_price using IQR method.give me the code

CODE:

stock_market_preprocessor.py > ...

```
1  import pandas as pd
2  import numpy as np
3  from typing import Tuple
4
5  def preprocess_stock_data(df: pd.DataFrame) -> pd.DataFrame:
6      """
7      Preprocesses a stock market dataset by handling missing values,
8      creating lag features, normalizing volume, and detecting outliers.
9
10     Args:
11         df (pd.DataFrame): The raw stock market DataFrame. Expected columns
12         | | | | | include 'Date', 'closing_price', and 'volume'.
13
14     Returns:
15         pd.DataFrame: The preprocessed DataFrame with new features and
16         | | | | | outlier flags.
17     """
18     processed_df = df.copy()
19
20     # Ensure 'Date' column is datetime and set as index for time-series operations
21     if 'Date' in processed_df.columns:
22         processed_df['Date'] = pd.to_datetime(processed_df['Date'])
23         processed_df.set_index('Date', inplace=True)
24     processed_df.sort_index(inplace=True) # Ensure data is sorted by date
25
26     # 1. Handle missing values in 'closing_price' and 'volume'
27     # For time-series data, forward-fill (ffill) is often a good strategy,
28     # followed by backward-fill (bfill) to catch any NaNs at the beginning.
29     print("Handling missing values...")
30     if 'closing price' in processed df.columns:
```

Ln 102, Col 62: Spaces: 4, LHS: 8, RHS: 1, Python 3.12

```

30     if 'closing_price' in processed_df.columns:
31         processed_df['closing_price'].fillna(method='ffill', inplace=True)
32         processed_df['closing_price'].fillna(method='bfill', inplace=True)
33         # If still NaN (e.g., all values were NaN), fill with median
34         processed_df['closing_price'].fillna(processed_df['closing_price'].median(), inplace=True)
35
36     if 'volume' in processed_df.columns:
37         processed_df['volume'].fillna(method='ffill', inplace=True)
38         processed_df['volume'].fillna(method='bfill', inplace=True)
39         # If still NaN, fill with median (or 0 if appropriate for volume)
40         processed_df['volume'].fillna(processed_df['volume'].median(), inplace=True)
41         # Ensure volume is non-negative before log scaling
42         processed_df['volume'] = processed_df['volume'].apply(lambda x: max(0, x))
43
44     # 2. Create lag features (1-day, 7-day returns)
45     print("Creating lag features (returns)...")
46     if 'closing_price' in processed_df.columns:
47         # 1-day return (percentage change from previous day)
48         processed_df['1_day_return'] = processed_df['closing_price'].pct_change(1) * 100
49         # 7-day return (percentage change from 7 days ago)
50         processed_df['7_day_return'] = processed_df['closing_price'].pct_change(7) * 100
51         # Fill NaNs created by pct_change (first few rows) with 0 or median
52         processed_df['1_day_return'].fillna(0, inplace=True)
53         processed_df['7_day_return'].fillna(0, inplace=True)
54
55     # 3. Normalize volume column using log-scaling
56     print("Normalizing volume using log-scaling...")
57     if 'volume' in processed_df.columns:
58         # Use np.log1p which computes log(1+x) to handle potential zero volumes gracefully

```



```

59     processed_df['volume_log_scaled'] = np.log1p(processed_df['volume'])
60
61     # 4. Detect outliers in 'closing_price' using IQR method
62     print("Detecting outliers in closing_price using IQR method...")
63     if 'closing_price' in processed_df.columns:
64         Q1 = processed_df['closing_price'].quantile(0.25)
65         Q3 = processed_df['closing_price'].quantile(0.75)
66         IQR = Q3 - Q1
67         lower_bound = Q1 - 1.5 * IQR
68         upper_bound = Q3 + 1.5 * IQR
69
70         processed_df['is_closing_price_outlier'] = (
71             (processed_df['closing_price'] < lower_bound) |
72             (processed_df['closing_price'] > upper_bound)
73         )
74
75     return processed_df
76
77 # --- Example Usage ---
78 if __name__ == "__main__":
79     # Sample stock market dataset
80     # Dates are important for lag features
81     sample_data = {
82         'Date': pd.to_datetime(['2023-01-01', '2023-01-02', '2023-01-03', '2023-01-04',
83                                '2023-01-05', '2023-01-06', '2023-01-07', '2023-01-08',
84                                '2023-01-09', '2023-01-10', '2023-01-11', '2023-01-12',
85                                '2023-01-13', '2023-01-14', '2023-01-15']),
86         'closing_price': [100.0, 101.5, 102.0, None, 103.5, 104.0, 105.0, 106.0,
87                           115.0, 107.0, 108.0, 109.0, 110.0, 111.0, 150.0], # Outlier at 150
88         'volume': [100000, 120000, None, 90000, 110000, 130000, 105000, 115000,
89                   140000, 95000, 125000, 10000, 135000, 100000, 160000] # Low volume at 10000
90     }
91     raw_df = pd.DataFrame(sample_data)
92
93     print("--- Original Raw Stock Data ---")
94     print(raw_df)
95     print("\n" + "="*70 + "\n")
96
97     # Preprocess the dataset
98     cleaned_df = preprocess_stock_data(raw_df)
99
100    print("\n--- Preprocessed Stock Data for Analysis ---")
101    print(cleaned_df)
102    print("\nSummary of outliers detected:")
103    print(cleaned_df[cleaned_df['is_closing_price_outlier']])

```

OUTPUT:

```
PS C:\Users\DELL\Desktop\vs code\.vscode> & C:/Users/DELL/AppData/Local/anaconda3/python.exe "c:/Users/DELL/Desktop/vs code/.vscode/stock_market_preprocessor.py"
```

```
--- Original Raw Stock Data ---
```

	Date	closing_price	volume
0	2023-01-01	100.0	100000.0
1	2023-01-02	101.5	120000.0
2	2023-01-03	102.0	NaN
3	2023-01-04	NaN	90000.0
4	2023-01-05	103.5	110000.0
5	2023-01-06	104.0	130000.0
6	2023-01-07	105.0	105000.0
7	2023-01-08	106.0	115000.0
8	2023-01-09	115.0	140000.0
9	2023-01-10	107.0	95000.0
10	2023-01-11	108.0	125000.0
11	2023-01-12	109.0	10000.0
12	2023-01-13	110.0	135000.0
13	2023-01-14	111.0	100000.0
14	2023-01-15	150.0	160000.0

```
=====
```

```
Handling missing values...
```

```
c:\Users\DELL\Desktop\vs code\.vscode\stock_market_preprocessor.py:31: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignment using an inplace method. The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values always behaves as a copy.
```

```
processed_df['7_day_return'].fillna(0, inplace=True)
```

```
Normalizing volume using log-scaling...
```

```
Detecting outliers in closing_price using IQR method...
```

```
--- Preprocessed Stock Data for Analysis ---
```

	Date	closing_price	volume	...	volume_log_scaled	is_closing_price_outlier
	2023-01-01	100.0	100000.0	...	11.512935	False
	2023-01-02	101.5	120000.0	...	11.695255	False
	2023-01-03	102.0	120000.0	...	11.695255	False
	2023-01-04	102.0	90000.0	...	11.407576	False
	2023-01-05	103.5	110000.0	...	11.608245	False
	2023-01-06	104.0	130000.0	...	11.775297	False
	2023-01-07	105.0	105000.0	...	11.561725	False
	2023-01-08	106.0	115000.0	...	11.652696	False
	2023-01-09	115.0	140000.0	...	11.849405	False
	2023-01-10	107.0	95000.0	...	11.461643	False
	2023-01-11	108.0	125000.0	...	11.736077	False
	2023-01-12	109.0	10000.0	...	9.210440	False
	2023-01-13	110.0	135000.0	...	11.813037	False
	2023-01-14	111.0	100000.0	...	11.512935	False
	2023-01-15	150.0	160000.0	...	11.982935	True

```
[15 rows x 6 columns]
```

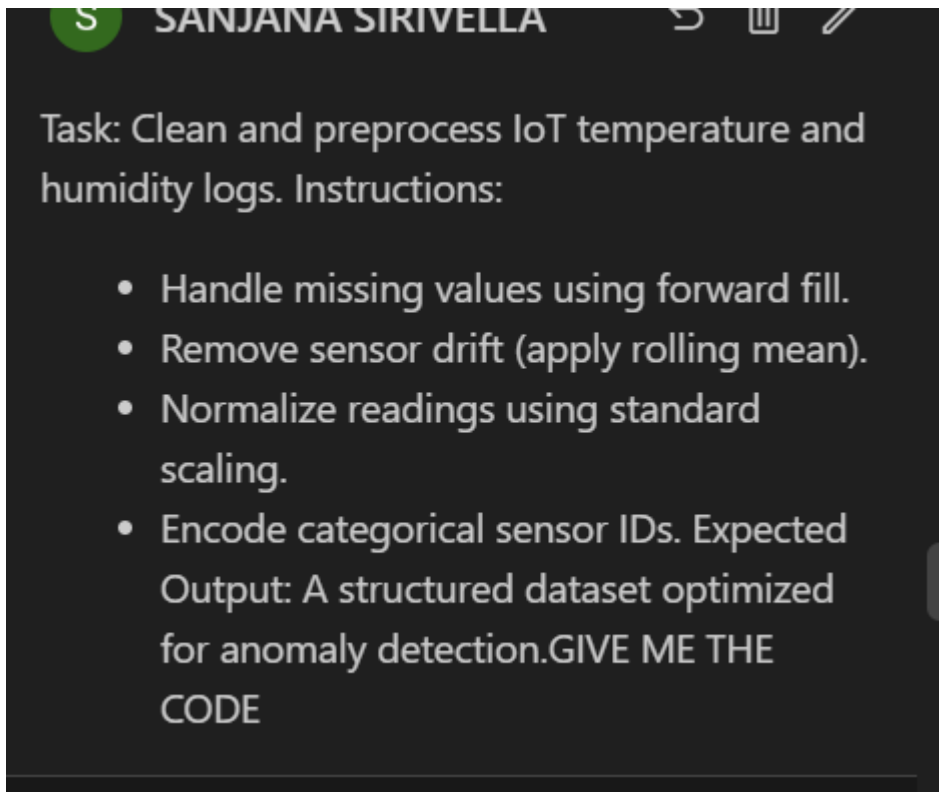
```
Summary of outliers detected:
```

```
Summary of outliers detected:
      closing_price  volume  ...  volume_log_scaled  is_closing_price_outlier
Date
2023-01-15      150.0  160000.0  ...      11.982935                True

[1 rows x 6 columns]
```

TASK:03

PROMPT:

A screenshot of a chat interface with a dark background. At the top, a green circular profile icon is next to the name "SANJANA SIRIVELLA". Below the name, the text "Task: Clean and preprocess IoT temperature and humidity logs. Instructions:" is displayed. Underneath, there is a bulleted list of four instructions. At the bottom of the list, the text "Output: A structured dataset optimized for anomaly detection.GIVE ME THE CODE" is written. On the right side of the chat bubble, there are three icons: a left-pointing arrow, a trash can, and a pencil.

SANJANA SIRIVELLA

Task: Clean and preprocess IoT temperature and humidity logs. Instructions:

- Handle missing values using forward fill.
- Remove sensor drift (apply rolling mean).
- Normalize readings using standard scaling.
- Encode categorical sensor IDs. Expected Output: A structured dataset optimized for anomaly detection.GIVE ME THE CODE

CODE:

iot_preprocessor.py > ...

```
1 import pandas as pd
2 import numpy as np
3
4 def preprocess_iot_data(df: pd.DataFrame, rolling_window: int = 3) -> pd.DataFrame:
5     """
6     Cleans and preprocesses IoT sensor data for anomaly detection.
7
8     Args:
9         df (pd.DataFrame): Raw IoT data with columns 'timestamp', 'sensor_id',
10            'temperature', and 'humidity'.
11         rolling_window (int): The window size for the rolling mean to smooth data.
12
13     Returns:
14         pd.DataFrame: A preprocessed DataFrame ready for machine learning.
15     """
16     processed_df = df.copy()
17
18     # Ensure timestamp is a datetime object and sort
19     processed_df['timestamp'] = pd.to_datetime(processed_df['timestamp'])
20     processed_df.sort_values(by=['sensor_id', 'timestamp'], inplace=True)
21
22     # 1. Handle missing values using forward fill within each sensor group
23     print("Handling missing values with forward fill...")
24     # Group by sensor to prevent filling data from one sensor to another
25     for col in ['temperature', 'humidity']:
26         processed_df[col] = processed_df.groupby('sensor_id')[col].transform(
```

```
27         lambda x: x.ffill().bfill()
28     )
29
30     # 2. Remove sensor drift/noise by applying a rolling mean
31     print(f"Applying a {rolling_window}-period rolling mean to smooth data...")
32     processed_df['temp_smoothed'] = processed_df.groupby('sensor_id')['temperature'].transform(
33         lambda x: x.rolling(window=rolling_window, min_periods=1).mean()
34     )
35     processed_df['humidity_smoothed'] = processed_df.groupby('sensor_id')['humidity'].transform(
36         lambda x: x.rolling(window=rolling_window, min_periods=1).mean()
37     )
38
39     # 3. Normalize readings using standard scaling (Z-score)
40     print("Normalizing readings with standard scaling...")
41     for col in ['temp_smoothed', 'humidity_smoothed']:
42         scaled_col_name = col.replace('_smoothed', '_scaled')
43         # Calculate mean and std dev per sensor
44         stats = processed_df.groupby('sensor_id')[col].agg(['mean', 'std'])
45         # Map stats back to the main dataframe
46         processed_df = processed_df.join(stats, on='sensor_id')
47         # Apply scaling
48         processed_df[scaled_col_name] = (processed_df[col] - processed_df['mean']) / processed_df[
49             # Clean up temporary columns
50             processed_df.drop(columns=['mean', 'std'], inplace=True)
51
52     # Handle cases where std is 0 (all values are the same), which results in NaN
53     processed_df.fillna(0, inplace=True)
54
```

```

55     # 4. Encode categorical sensor IDs using one-hot encoding
56     print("Encoding categorical sensor IDs...")
57     sensor_dummies = pd.get_dummies(processed_df['sensor_id'], prefix='sensor')
58     processed_df = pd.concat([processed_df, sensor_dummies], axis=1)
59
60     # Final cleanup for the model-ready dataset
61     # Drop original and intermediate columns, keep only the processed features
62     final_cols = ['timestamp'] + [col for col in processed_df.columns if '_scaled' in col or 'sensor' in col]
63     model_ready_df = processed_df[final_cols].copy()
64
65     # Drop any rows that might have NaNs from the initial rolling window
66     model_ready_df.dropna(inplace=True)
67
68     return model_ready_df.reset_index(drop=True)
69
70 # --- Example Usage ---
71 if __name__ == "__main__":
72     # Sample raw IoT sensor data
73     iot_data = {
74         'timestamp': pd.to_datetime([
75             '2023-01-01 10:00:00', '2023-01-01 10:01:00', '2023-01-01 10:02:00',
76             '2023-01-01 10:03:00', '2023-01-01 10:04:00', '2023-01-01 10:00:00',
77             '2023-01-01 10:01:00', '2023-01-01 10:02:00', '2023-01-01 10:03:00',
78             '2023-01-01 10:04:00'
79         ]),
80         'sensor_id': ['A-101', 'A-101', 'A-101', 'A-101', 'A-101',
81                     'B-202', 'B-202', 'B-202', 'B-202', 'B-202'],
82         'temperature': [22.1, 22.3, None, 22.4, 25.0, 35.5, 35.6, 35.8, None, 36.0], # Includes missing values
83         'humidity': [65.2, 65.1, 65.3, None, 65.4, 70.1, None, 70.5, 70.6, 70.8] # Includes missing values

```

```

84     }
85     raw_df = pd.DataFrame(iot_data)
86
87     print("--- Original Raw IoT Dataset ---")
88     print(raw_df)
89     print("\n" + "="*70 + "\n")
90
91     # Preprocess the dataset
92     preprocessed_df = preprocess_iot_data(raw_df)
93
94     print("\n--- Preprocessed Dataset for Anomaly Detection ---")
95     print(preprocessed_df)

```

OUTPUT:

```
PS C:\Users\DELL\Desktop\vs code\.vscode> & C:/Users/DELL/AppData/Local/anaconda3/python.exe "c:/Users/DELL/Desktop/vs code/.vscode/iot_preprocessor.py"
--- Original Raw IoT Dataset ---
    timestamp sensor_id temperature humidity
0 2023-01-01 10:00:00 A-101      22.1      55.2
1 2023-01-01 10:01:00 A-101      22.3      55.1
2 2023-01-01 10:02:00 A-101      NaN      55.3
3 2023-01-01 10:03:00 A-101      22.4      NaN
4 2023-01-01 10:04:00 A-101      25.0      55.4
5 2023-01-01 10:00:00 B-202      35.5      70.1
6 2023-01-01 10:01:00 B-202      35.6      NaN
7 2023-01-01 10:02:00 B-202      35.8      70.5
8 2023-01-01 10:03:00 B-202      NaN      70.6
9 2023-01-01 10:04:00 B-202      36.0      70.8

=====

Handling missing values with forward fill...
Applying a 3-period rolling mean to smooth data...
Normalizing readings with standard scaling...
Encoding categorical sensor IDs...

--- Preprocessed Dataset for Anomaly Detection ---
    timestamp sensor_id temp_scaled humidity_scaled sensor_A-101 sensor_B-202
0 2023-01-01 10:00:00 A-101  -0.692280      -0.341565         True      False
1 2023-01-01 10:01:00 A-101  -0.475942      -1.073490         True      False
2 2023-01-01 10:02:00 A-101  -0.403830      -0.341565         True      False
3 2023-01-01 10:03:00 A-101  -0.187492       0.146385         True      False
4 2023-01-01 10:04:00 A-101   1.759544       1.610235         True      False
5 2023-01-01 10:00:00 B-202  -1.065708      -0.853310        False      True
6 2023-01-01 10:01:00 B-202  -0.725588      -0.853310        False      True
7 2023-01-01 10:02:00 B-202  -0.158722      -0.264820        False      True
8 2023-01-01 10:03:00 B-202   0.521516       0.470792        False      True
9 2023-01-01 10:04:00 B-202   1.428502       1.500649        False      True
PS C:\Users\DELL\Desktop\vs code\.vscode>
```

TASK:04

PROMPT:

S SANJANA SIRIVELLA

ask: A streaming platform wants to analyze customer reviews. Instructions:

- Standardize text (lowercase, remove HTML tags).
- Tokenize and encode reviews using AI-assisted methods (TF-IDF or embeddings).
- Handle missing ratings (fill with median).
- Normalize ratings (0–10 → 0–1 scale).
- Generate a before vs after summary report.GIVE ME THE CODE

CODE:

review_preprocessor.py > ...

```
1 import pandas as pd
2 import re
3 from sklearn.feature_extraction.text import TfidfVectorizer
4 from typing import Tuple
5
6 def generate_summary_report(df: pd.DataFrame, title: str):
7     """Generates and prints a summary report for a DataFrame."""
8     print(f"\n--- {title} ---")
9     print("\n[First 5 Rows]")
10    print(df.head())
11
12    print("\n[Missing Values per Column]")
13    print(df.isnull().sum())
14
15    print("\n[Data Types]")
16    print(df.info(verbose=False))
17
18    print("\n[Descriptive Statistics for Numerical Columns]")
19    # Select only numeric columns for describe()
20    numeric_cols = df.select_dtypes(include=['number'])
21    if not numeric_cols.empty:
22        print(numeric_cols.describe())
23    else:
24        print("No numerical columns to describe.")
25    print("\n" + "="*60 + "\n")
26
27 def preprocess_customer_reviews(df: pd.DataFrame, max_tfidf_features: int = 50) -> pd.DataFrame:
28     """
29     Cleans, preprocesses, and encodes customer review data.
30
```



```

review_preprocessor.py > ...
27 def preprocess_customer_reviews(df: pd.DataFrame, max_tfidf_features: int = 50) -> pd.DataFrame:
30
31     Args:
32         df (pd.DataFrame): Raw review data with 'review_text' and 'rating' columns.
33         max_tfidf_features (int): The maximum number of TF-IDF features to generate.
34
35     Returns:
36         pd.DataFrame: A processed DataFrame with cleaned text and encoded features.
37     """
38     # --- Generate Before Report ---
39     generate_summary_report(df, "Before Preprocessing Summary")
40
41     processed_df = df.copy()
42
43     # 1. Handle missing ratings by filling with the median
44     median_rating = processed_df['rating'].median()
45     processed_df['rating'].fillna(median_rating, inplace=True)
46
47     # 2. Normalize ratings from 0-10 scale to 0-1 scale
48     processed_df['rating_normalized'] = processed_df['rating'] / 10.0
49
50     # 3. Standardize text (lowercase, remove HTML tags)
51     def clean_text(text: str) -> str:
52         if not isinstance(text, str):
53             return ""
54         text = text.lower() # Convert to lowercase
55         text = re.sub('<[^\>]+>', '', text) # Remove HTML tags
56         text = re.sub(r'[^\w\s]', '', text) # Remove punctuation and numbers
57         return text.strip()
58

```

```

review_preprocessor.py > ...
27 def preprocess_customer_reviews(df: pd.DataFrame, max_tfidf_features: int = 50) -> pd.DataFrame:
59     processed_df['cleaned_text'] = processed_df['review_text'].apply(clean_text)
60
61     # 4. Tokenize and encode reviews using TF-IDF
62     vectorizer = TfidfVectorizer(
63         max_features=max_tfidf_features,
64         stop_words='english'
65     )
66     tfidf_features = vectorizer.fit_transform(processed_df['cleaned_text'])
67
68     # Create a DataFrame from the TF-IDF features
69     tfidf_df = pd.DataFrame(
70         tfidf_features.toarray(),
71         columns=vectorizer.get_feature_names_out()
72     )
73
74     # Concatenate the TF-IDF features with the processed DataFrame
75     final_df = pd.concat([processed_df.reset_index(drop=True), tfidf_df], axis=1)
76
77     # --- Generate After Report ---
78     generate_summary_report(final_df, "After Preprocessing Summary")
79
80     return final_df
81
82 # --- Example Usage ---
83 if __name__ == "__main__":
84     # Sample raw customer review data for a streaming platform
85     review_data = {
86         'review_id': [1, 2, 3, 4, 5, 6],
87         'review_text': [

```

```

87         'review_text': [
88             'This show is AMAZING! <br>I loved every minute of it.',
89             'The plot was very slow and boring.',
90             'Great acting, but the ending was predictable.',
91             '<b>Absolutely terrible</b>. A complete waste of time.',
92             'A masterpiece of modern cinema. The best movie!',
93             'Good, but not great. Just an average movie.'
94         ],
95         'rating': [10, 3, 7, 1, None, 6] # Includes a missing value
96     }
97     raw_df = pd.DataFrame(review_data)
98
99     # Run the preprocessing pipeline
100     # The function will print the before/after reports
101     processed_data = preprocess_customer_reviews(raw_df, max_tfidf_features=10)
102
103     print("--- Final Processed DataFrame Head ---")
104     print(processed_data.head())

```

CODE:

```
PS C:\Users\DELL\Desktop\vs code\.vscode> & C:/Users/DELL/AppData/Local/anaconda3/python.exe "c:/Users/DELL/Desktop/vs code/.vscode/iot_preprocessor.py"
```

```
--- Original Raw IoT Dataset ---
```

	timestamp	sensor_id	temperature	humidity
0	2023-01-01 10:00:00	A-101	22.1	55.2
1	2023-01-01 10:01:00	A-101	22.3	55.1
2	2023-01-01 10:02:00	A-101	NaN	55.3
3	2023-01-01 10:03:00	A-101	22.4	NaN
4	2023-01-01 10:04:00	A-101	25.0	55.4
5	2023-01-01 10:00:00	B-202	35.5	70.1
6	2023-01-01 10:01:00	B-202	35.6	NaN
7	2023-01-01 10:02:00	B-202	35.8	70.5
8	2023-01-01 10:03:00	B-202	NaN	70.6
9	2023-01-01 10:04:00	B-202	36.0	70.8

```
=====  
Handling missing values with forward fill...  
Applying a 3-period rolling mean to smooth data...  
Normalizing readings with standard scaling...  
Encoding categorical sensor IDs...
```

```
--- Preprocessed Dataset for Anomaly Detection ---
```

	timestamp	sensor_id	temp_scaled	humidity_scaled	sensor_A-101	sensor_B-202
0	2023-01-01 10:00:00	A-101	-0.692280	-0.341565	True	False
1	2023-01-01 10:01:00	A-101	-0.475942	-1.073490	True	False
2	2023-01-01 10:02:00	A-101	-0.403830	-0.341565	True	False
3	2023-01-01 10:03:00	A-101	-0.187492	0.146385	True	False
4	2023-01-01 10:04:00	A-101	1.759544	1.610235	True	False

```
--- Preprocessed Dataset for Anomaly Detection ---
```

	timestamp	sensor_id	temp_scaled	humidity_scaled	sensor_A-101	sensor_B-202
0	2023-01-01 10:00:00	A-101	-0.692280	-0.341565	True	False
1	2023-01-01 10:01:00	A-101	-0.475942	-1.073490	True	False
2	2023-01-01 10:02:00	A-101	-0.403830	-0.341565	True	False
3	2023-01-01 10:03:00	A-101	-0.187492	0.146385	True	False
4	2023-01-01 10:04:00	A-101	1.759544	1.610235	True	False
5	2023-01-01 10:00:00	B-202	-1.065708	-0.853310	False	True
6	2023-01-01 10:01:00	B-202	-0.725588	-0.853310	False	True
7	2023-01-01 10:02:00	B-202	-0.158722	-0.264820	False	True
8	2023-01-01 10:03:00	B-202	0.521516	0.470792	False	True
9	2023-01-01 10:04:00	B-202	1.428502	1.500649	False	True

```
PS C:\Users\DELL\Desktop\vs code\.vscode> & C:/Users/DELL/AppData/Local/anaconda3/python.exe "c:/Users/DELL/Desktop/vs code/.vscode/review_preprocessor.py"
```

```
--- Before Preprocessing Summary ---
```

```
[First 5 Rows]
```

	review_id	review_text	rating
0	1	This show is AMAZING! I loved every minute...	10.0
1	2	The plot was very slow and boring.	3.0
2	3	Great acting, but the ending was predictable.	7.0
3	4	Absolutely terrible. A complete waste o...	1.0
4	5	A masterpiece of modern cinema. The best movie!	NaN

```
[Missing Values per Column]
```

review_id	0
review_text	0
rating	1

-----THE END-----